

D8/T8 Contactless Smart Card Reader
Reference manual

The D8 includes RS232 and USB port connections. It is used as a contactless smart card reader in generally, but it can be expanded dual Mode Smart Card Reader. Meantime D8can adds 2-3 SAM cards for choice of consumer. It includes an antenna and a buzzer. All the Mifare cards can be read and written. The ISO 14443 type A and B cards can also be accessed on request. It is mainly used in parking-meter, door-control, Health care—medical identification, transportation, oil-control, access control identifications.

Technical specifications

Product Name	D8 Contactless Smart Card Reader
weight	300g
Storage range	-20 to +60°C
Humidity range	(95±2)%
Power supply voltage	5V , powered by USB or keyboard port
Power supply current	<150mA
Communication type with PC	USB no_driven or RS232
Following standard	ISO14443、ISO15693、ISO 7816、GSM11.11、FCC、CE
Support contactless card type ⁽¹⁾	Support ISO14443 TypeA/B protocol contactless card Support ISO15693 protocol contactless card card type: Mifare S50、Mifare S70、MF1ICL10、Mifare Pro、Mifare desfire、Mifare ultralight、SLE44R31、SLE66CL series、TYPEB card、SHC1102、FM11RF005、 FM11RF32、Mifare Icode II、Ti 2K and so on.
Contact card ⁽²⁾	Support 1 followed ISO7816 standard card size Support 3 GSM 11.11 _followed SIM card size Support ISO7816 protocol T0、T1_followed card and general storage card,logic_encryption card.
Contact card's contact spot using time	100 thousand times
Contact card supply current	0-130mA
Contact card BPS ⁽³⁾	T=0、T=1: 9600—38400bps
Status display	LED indicator light, points to the electricity and communication status
size	123mm*95mm*27mm
Operation systems ⁽⁴⁾	Windows 98、Me、2K、XP、2003 and Linux、Unix、Dos
Other characteristic	Supply general interface function ,support kinds of operation systems and languages, Also supply the example programme that using various languages. Hardware supports upgrade online

(1) D8 general version only support ISO-14443 TYPEA protocol card, please note in advance if you need reader that support other types of card

(2) D8 general version mainly support ISO7816 protocol T0、T1 card。It need special type to support general storage card,logic_encryption card.

(3) To T0、T1 CPU card ,the supporting baudrate can be improved,up to 115200bps。

(4) Unix,DOS operation system need special type to support

Reader driver function introduction

Function Overview

General Function:

1. dc_init
2. dc_exit
3. dc_card
4. dc_request
5. dc_anticoll
6. dc_select
7. dc_load_key
8. dc_authentication
9. dc_read
10. dc_write
11. dc_halt
12. dc_des
13. dc_changeb3
14. dc_authentication_passaddr
15. dc_initval
16. dc_increment
17. dc_readval
18. dc_decrement
19. dc_HL_authentication
20. dc_HL_read
21. dc_HL_write
22. dc_restore
23. dc_transfer
24. dc_authentication_2
25. dc_authentication_pass
26. dc_card_double
27. dc_card_str

Device Function:

1. dc_beep
2. dc_disp_mode
3. dc_disp_str
4. dc_gettime
5. dc_getver
6. dc_high_disp
7. dc_setbright
8. dc_settime
9. dc_srd_eeprom
10. dc_swr_eeprom
11. a_hex

12. hex_a
13. dc_reset

SHC1102 Card Special Function:

1. dc_request_shc1102
2. dc_auth_shc1102
3. dc_read_shc1102
4. dc_write_shc1102

Mifare Ultralight Special Function:

1. dc_anticoll2
2. dc_select2

ISO14443-4 Card Special Function:

1. dc_config_card
2. dc_pro_command
3. dc_pro_commandsource
4. dc_pro_commandlink
5. dc_pro_reset(Type A specialized)
6. dc_pro_halt(Type A specialized)
7. dc_request_b(Type B specialized)
8. dc_attrib(Type B specialized)
9. dc_card_b
10. dc_cardAB

Mifare Des Card Special Function:

1. dc_anticoll2
2. dc_select2
3. dc_pro_reset
4. dc_pro_halt
5. dc_pro_command
6. dc_pro_commandsource
7. dc_pro_commandlink

CPU(SAM) Card Special Function:

1. dc_cpureset
2. dc_setcpu
3. dc_cpuapdu
4. dc_cpuapdusource
5. dc_setcpupara

FM11RF005 Card Special Function:

1. dc_read_fm11rf005
2. dc_write_fm11rf005

3. dc_getsnr_fm11rf005

T8 LCD Display Function:

1. dc_dispinfo_T8
2. dc_dispinfo_pro_T8
3. dc_clearlcd_T8

Function Details

General Function:

1) HANDLE dc_init(int port,long baud);

Description: This function initializes the port.

Parameters: :

port: com number(0~3) if port=100 USB

baud: baudrate(9600~115200)

Return: >0: correct;

Example: HANDLE icdev;

icdev=dc_init(1,115200); /baudrate:115200,port:com2/

2) int dc_exit(HANDLE icdev);

Description: exit the port

Parameter:

icdev: the communication identifier of the device

Return: =0: correct

<>0: error

Example: int st;

st=dc_exit(icdev);

3) int dc_card(HANDLE icdev,unsigned char _Mode,unsigned long *_Snr);

Description: request card and return the card's serial number (This function includes the function dc_request, dc_antirecoll, dc_select)

Parameters

icdev: the communication identifier of the device

_Mode: request card mode

Mode=0: IDLE mode

Mode=1: ALL mode

_Snr: returns the card's serial number

Return: =0 : correct

<>0: error

Example: int st;

unsigned char Mode=0; //IDLE mode

unsigned long snr;

st=dc_card(icdev,Mode,&snr);

Attention: Instruction about find card mode :

If authentication the card with the IDLE mode when finding card,, after finishi

ng the operation and call the function `dc_halt` to stop the operation, then the reader couldn't find this card again, unless this card is taken out of the operating area and put into again.

If authentication with the ALL mode when finding card, after finishing the operation and call function `dc_halt` to stop the operation, the reader could always find the card again. Needn't take out it and put in the operating area again .

4) `int dc_request(HANDLE icdev, unsigned char _Mode, unsigned int *TagType);`

Description: This function sends the request command to the card

Parameters:

`icdev`: the communication identifier of the device

`_Mode`: find card mode

Mode=0: IDLE mode

Mode=1: ALL mode

`Tagtype`: returns card's type

Return: =0: correct

<>0: error

Example:

```
unsigned char Mode=0;
unsigned int tagtype;
st=dc_request(icdev,Mode,&tagtype);
```

5) `int dc_antirecoll(HANDLE icdev, unsigned char _Bcnt, unsigned long *_Snr);`

Description: anti-collision, returns the serial number of the card

Parameters

`icdev`: the communication identifier of the device

`_Bcnt`: the number of used bits for the pre-selection of the cards. standard call is `bcnt=0`

`_Snr`: returns the serial number of the card

Return: = 0: correct

<>0: error

Example: `int st;`

```
unsigned long snr;
st=dc_antirecoll(icdev,0,&snr);
```

Note: After execute the `dc_request` ,you must use the `dc_antirecoll` immediately, unless the serial number of the card is known.

6) `int dc_select(HANDLE icdev, unsigned long _Snr, unsigned char *_Size);`

Description: selects the card with the specified serial number, It returns the size of the memory of the card to PC

Parameters

`Icdev`: the communication identifier of the device

`_Snr`: serial number of the card

`_Size`: the memory size of the card. The expected size of the MIFARE1 card is 0x08(8kbit).

Return: = 0: correct

<>0: error

Example: int st;

unsigned long snr=239474;

unsigned char size;

dc_request(icdev,0,&type);

dc_anticoll(icdev,0,&snr);

st=dc_select(icdev,snr,&size);

**7) int dc_load_key(HANDLE icdev,unsigned char _Mode,unsigned char SecNr,
unsigned char * _Nkey);**

Description: load key to Reader

Parameters

icdev: the communication identifier of the device

_Mode: authentication key mode:

0 — authentication with the 0th KEY A

1 — authentication with the 1th KEY A

2 — authentication with the 2th KEY A

4 — authentication with the 0th KEY B

5 — authentication with the 1th KEY B

6 — authentication with the 2th KEY B

_SecNr: the sector that to authentication(0~15)

_Nkey: the data that write into the Reader's RAM

Return: = 0: correct

<>0: error

Example: //load the 0th key A for sector 1

unsigned char key[6]= {0xa0,0xa1,0xa2,0xa3,0xa4,0xa5 };

st=dc_load_key(icdev,0,1,key);

Related Function:

__int16 __stdcall dc_load_key_hex(HANDLE icdev,unsigned char _Mode,unsigned char
_SecNr,unsigned char * _Nkey

**8) int dc_authentication(HANDLE icdev ,unsigned char _Mode,
unsigned char _SecNr);**

Description: authentication one sector

Parameters

icdev: the communication identifier of the device

_Mode: authentication key mode:

0 — authentication with the 0th KEY A

1 — authentication with the 1th KEY A

2 — authentication with the 2th KEY A

4 — authentication with the 0th KEY B

5 — authentication with the 1th KEY B

6 — authentication with the 2th KEY B

_SecNr: the sector that authentication(0~15)

Return: = 0: correct

<>0: error

Example: int st;

st=dc_authentication(icdev,0,5); //authentication the 5 sector with the 0th key

A

9) int dc_read(HANDLE icdev,unsigned char _Adr,unsigned char *_Data);

Description: this function reads one block of 16 bytes from a selected and authenticated card

Parameters

icdev: the communication identifier of the device

_Adr: block address from which the data is read(0~63)

_Data: stores data that read from the card. The memory space in the RAM of the PC must be allocated from the calling function.

Return: = 0: correct

<>0: error

Example: int st;

unsigned char data[16];

st=dc_read(icdev,1,data);

Related Function:

__int16 __stdcall dc_read_hex(HANDLE icdev,unsigned char _Adr,char *_Data)

10) int dc_write(HANDLE icdev,unsigned char _Adr,unsigned char *_Data);

Description: this function writes one block of 16 bytes to a selected and authenticated card. Back-up management and security schemes must be implemented in the system software.

Parameters:

icdev: the communication identifier of the device

_Adr: block address(1~63)

_Data: stores data that written into the card, length is 16.

Return: =0: correct

<>0: error

Example: int st;

unsigned char data[16]={0x00,0x11,0x22,0x33,0x44,0x55,0x66,0x77,0x88,0x88,0x77,0x66,0x55,0x44,0x33,0x22,0x11};

st=dc_write(icdev,1,data); //write block 1

Related Function:

__int16 __stdcall dc_write_hex(HANDLE icdev,unsigned char _Adr,char *_Data)

11) int dc_halt(HANDLE icdev);

Description: this function sets a selected tag into "halt mode", afterwards it does not engaged in any communication until it is reset again or a request (ALL) function is called.

Parameters

icdev: the communication identifier of the device

Return: =0: correct

<>0: error

Example: st=dc_halt(icdev);

Attention : use this function after each operation.

12) int dc_des(unsigned char *key,unsigned char *sour,unsigned char *dest,__int16 m)

Description: DES Algorithm Encryption and Decryption Function

Parameters:

key: key

sour: the data that to encrypted or decrypted

dest: the data after encryption

m: encryption-decryption Mode

m=1:encryption

m=0:decryption

Return:=0 correct

Related Function:

__int16 __stdcall dc_des_hex(unsigned char *key,unsigned char *sour,unsigned char *dest, __int16 m);

13) int dc_changeb3(HANDLE icdev,unsigned char _SecNr,unsigned char *_KeyA,unsigned char _B0,unsigned char _B1,unsigned char _B2,unsigned char _B3,unsigned char _Bk,unsigned char *_KeyB);

Description: update data of the third block

Parameters

icdev: the communication identifier of the device

_SecNr: sector (0~15)

_KeyA: key A

_B0: the control bits of 0th block. low three bit (D2D1D0) correspondence
C10, C20,C30

_B1: the control bits of 1th block. low three bit (D2D1D0) correspondence
C11, C21,C31

_B2: the control bits of 0th block. low three bit (D2D1D0) correspondence
C12, C22,C32

_B3: the control bits of 0th block. low three bit (D2D1D0) correspondence
C13, C23,C33

_Bk: Reservation parameters: set 0

_KeyB: key B

Return: =0: correct

<>0: error

Example: int st;

unsigned char keyA[6]={0xa0,0xa1,0xa2,0xa3,0xa4,0xa5};

unsigned char keyB[6]={0xb0,0xb1,0xb2,0xb3,0xb4,0xb5};

st=dc_changeb3(icdev,keya,0x04,0x04,0x04,0x04,0,keyb);

14) **__int16 __stdcall dc_authentication_passaddr(HANDLE icdev,unsigned char _Mode, unsigned char blockAddr, unsigned char *passbuff)**

Description: authentication function, when using this function, the dc_load_key() needn't executed.

Parameter:

icdev: the communication identifier that dc_init returns

_Mode: authentication Mode

blockAddr: the Block Address that to authenticated

passbuff: password string

Return: =0 correct

Related Function:

__int16 __stdcall dc_authentication_passaddr_hex(HANDLE icdev,unsigned char _Mode, unsigned char blockAddr, unsigned char *passbuff)

15) int dc_initval(HANDLE icdev,unsigned char _Adr,unsigned long _Value);

Description: initializes the value block

Parameters

icdev: the communication identifier of the device

_Adr: block address(1~63)

_Value: the value to initialize

Return: =0: correct

<>0: error

Example: int st;

unsigned long value=1000;

st=dc_initval(icdev,1,value);

Attention: When value_operating the card, use the special data stucture, so t he block must be initialized at first, then can do some other read, increment , decrement operation.

16) int dc_increment(HANDLE icdev,unsigned char _Adr, unsigned long _Value);

Description: this function reads the accesseed value block,checks the data stucture, increase the contents of the value block by the transmitted value and stores the result in the card's internal register. Non writing operation to the card's EEPROM is performed

Parameters

icdev: the communication identifier of the device

_Adr: the address of the card that the value block to be increment(1~63, except the Key Block)

_Value: the value to increase

Return: = 0: correct

<>0: error

Example: int st;

unsigned long value=2;

st=dc_increment(icdev,1,value

17) int dc_readval(HANDLE icdev,unsigned char _Adr,unsigned long*_Value);

Description: read the value of the selected block.

Parameters:

icdev: the communication identifier of the device

_Adr: block address(0~63)

_Value: return the content of the value

Return: = 0: correct

<>0: error

Example: int st;

unsigned long value;

st=dc_readval(icdev,1,&value); //read the content and put in value

18) int dc_decrment(HANDLE icdev,unsigned char _Adr,unsigned long _Value);

Description: this function decreased the contents of the value block by the transmitted.

Parameters:

icdev: the communication identifier of the device

_Adr: the address of the card that the value block to be decrement(1~63, except the Key Block)

_Value: the value to decrease

Return: =0: correct

<>0: error

Example: int st;

unsigned long value=2;

st=dc_decrement(icdev,1,value);

19) int dc_HL_authentication(HANDLE icdev, unsigned char reqmode,unsigned long snr,unsigned char authmode,unsigned char secnr);

Description: high-level authentication (compose of dc_card() and dc_authentication())

Parameters

icdev: the communication identifier of the device

reqmode: find card mode, as dc_HL_initval()

snr: the serial number of card (available only in mode 2)

authmode: authentication key mode:

0 — authentication with the 0th KEY A

1 — authentication with the 1th KEY A

2 — authentication with the 2th KEY A

4 — authentication with the 0th KEY B

5 — authentication with the 1th KEY B

6 — authentication with the 2th KEY B

secnr: sector number (0~15)

Return: =0: correct

<>0: error

Example: //use IDLE mode

```
unsigned long snr;  
st=dc_HL_authentication(icdev,0,snr,0,3);
```

**20) int dc_HL_read(HANDLE icdev,unsigned char _Mode,unsigned char _Adr,
unsigned long _Snr,unsigned char *_Data,unsigned long *_NSnr);**

Description: high-level read function. This function reads one block of 16 BYTE to a selected and authenticated card. (use for block)

Parameters:

icdev: the communication identifier of the device
_Mode: find card mode,the same as dc_HL_initval()
_Adr: block address (1~63)
_Snr: the serial number of card (available only in mode 2)
_Data: stores the data read from the card(the length is 16)
_NSnr: return card's serial number

Return: = 0: correct

<>0: error

Example: unsigned long Snr,NSnr;

```
unsigned char data[16];  
st=dc_HL_read(icdev,0,3,Snr,data,&NSnr);
```

**21) int dc_HL_write(HANDLE icdev,unsigned char _Mode,unsigned char
_Adr,unsigned long *_Snr,unsigned char *_Data);**

Description: high-level write function.this function writes one block of 16 BYTE to a selected and authenticated card. (use for block)

Parameters

icdev: the communication identifier of the device
_Mode: find card mode, the same as dc_HL_initval()
_Adr: block address (1~63)
_Snr: the serial number of card (available only in mode 2)
_Data: stores the data written on the card(the length is 16)

Return: =0 correct

<>0 error

Example: unsigned long Snr;

```
unsigned char data[16]="f1f2f3f4f5f6f7f8";  
st=dc_HL_write(icdev,0,3,&Snr,data);
```

22) int dc_restore(HANDLE icdev,unsigned char _Adr);

Description: this function reads the accessed value block,checks the datastructure and stores the result in the card's internal register.Non writing operation to the card's EEPROM is performed

Parameters:

icdev: the communication identifier of the device
_Adr: the address of the card that the value block to be restored(1~63,ex

cept the Key Block)

Return: =0: correct

<>0: error

Example: int st;

st=dc_restore(icdev,1);

Note: This function is used to transfer the data of one block to the internal register. and then use the function dc_truansfer() transfers data of the internal register to a nother block of card. In this way the data can be transfered from one block to an other block.

23) int dc_transfer(HANDLE icdev,unsigned char _Adr);

Description: This function transfers the contents of the internal register to the trans mitted address.The sector must be authenticated for this operation .The transfer fu nction can only be called directly after increment,decrement or restore.

Parameters:

icdev: the communication identifier of the device

_Adr: the address on the card to which the contents of the internal regist er is transferred to (1~63)

Return: =0 correct

<>0: error

Example: int st;

st=dc_transfer(icdev,1);

24) int dc_authentication_2(HANDLE icdev, unsigned char _Mode,unsigned char KeyNr,unsigned char Adr);

Description: this function use for ML card to authentication

Parameters:

icdev: the communication identifier of the device

Mode: authentication key mode , the same as dc_authentication

KeyNr: =0

Adr: =0

Return: = 0: correct

<>0: error

Example: int st;

st=dc_authentication_2(icdev,1,0,0); //authentication with the 1th key A .

25) int dc_authentication_pass(int icdev,unsigned char _Mode,unsigned char Addr,unsigned char *passbuff)

Description: authentication one sector with input pass

Parameters:

icdev: the communication identifier of the device

_Mode: authentication mode

Adr: block address (1~63)

passbuff: 6 bytes pass

Return: =0: correct

<>0 error

Example:

```
unsigned char databuff[6]={0x00,0x11,0x22,0x33,0x44,0x55};
st= dc_authentication_pass (icdev,0,4,databuff); // write the fourth block
```

26) int dc_card_double(int icdev,unsigned char _Mode,unsigned char *_Snr);

Description: find card, the difference with [dc_card](#) is that it also use the dc_anticol2, dc_select2

Parameter:

icdev: the communication identifier of the device

_Mode: mode when finding card

_Snr: the serial number of the card (8 bytes)

Return: =0 correct

Example: int st;

```
unsigned char snr[8];
st=dc_card_double(icdev,0,snr);
```

Attention:If authentication the card with the IDLE mode when finding card,, after finishing the operation and call the function dc_halt to stop the operation, then the reader couldn't find this card again, unless this card is taken out of the operating area and put into again.

Related Function:

```
__int16 __stdcall dc_card_double_hex(HANDLE icdev,unsigned char
_Mode,unsigned char *snrstr
```

27) __int16 __stdcall dc_cardstr(HANDLE icdev,unsigned char _Mode,char * Strsnr);

Description: request card, returns card's serial number

Parameters:

icdev-- the communication identifier of the device

_Mode—request card mode,same as the Mode of dc_card IN

Mode=0 :IDLE mode

Mode=1:ALL mode

* Strsnr—card serial number (String) OUT

Return:

0--success

1—NO card

others--error

Notice: This function is almost the same as dc_card, the difference is that the serial number of dc_card that returns is long ,but dc_cardstr that returns is unsigned char .

Device Function:

1) int dc_beep(HANDLE icdev,unsigned int _Msec);

Description: beep

Parameters:

icdev: the communication identifier of the device
 _Msec: the time of beep, (ms)
 Return: = 0: correct
 <>0: error
 Example: st=dc_beep(icdev,10); //beep 100ms

2) int dc_disp_mode(int icdev,unsigned char mode);

Description: set the Nixietub's display mode of the Reader,it can save the setting information after closing down the PC

Parameter:

icdev: the communication identifier of the device
 mode: display mode
 0——date, the format is “year-month-day (yy-mm-dd) ”
 1——time, the format is “hour-minute-second (hh-nn-ss) ”

Return: =0 correct

Example: st=dc_disp_mode(icdev,0x01);//set as the display time

3) int dc_disp_str(int icdev,char *digit);

Description:display the data on the Reader's nixietube (RD800M Reader specialized)

Parameter:

icdev: the communication identifier of the device
 digit: the string that to display, it's composed of '0'-'F'and'.'

Return: =0 correct

Example: int st;
 st=dc_disp_str(icdev,"1.234"); /*show 1.234*/

4) int dc_gettime(int icdev,unsigned char *time);

Description:read the date,week,time of the Reader

Parameter:

icdev: the communication identifier of the device
 time: store data that returns, the length is 7 bytes, the format is “year、week、month、day、hour、minute、second”

Return: =0 correct

Example: int st;
 unsigned char datetime[7];
 st=dc_gettime(icdev,datetime);
 //datetime is “0x04,0x01,0x04,0x19,0x17,0x23,0x10”,
 //means that year 04 ,Monday, April, 19th,17:00,23m,10s

Related Function:

__int16 __stdcall dc_gettimehex(HANDLE icdev,char *time)

5) int dc_getver(HANDLE icdev,unsigned char *_buffer);

Description: get the Reader's version

Parameters

icdev: the communication identifier of the device
 _buffer: stores the device's version number,3 bytes

Return: =0: correct
<>0: error

Example: int st;
unsigned char buffer[5];
st=dc_getver(icdev,buffer);

6) int dc_high_disp(int icdev,unsigned char offset,unsigned char disp_len,unsigned char *disp_str);

Description: digital display

Parameter:

icdev: the communication identifier of the device

offset: the offset of the string that to display, range (1-8)

disp_len: the length of the string that to display, maximum 8,when length is 0,that's clear the screen.

disp_str: the string that to display. When the high half_byte is 8,the decimal point is bright; when 0,the decimal point is darkness.

Return: =0 correct

Example:int st;
unsigned char *numbuff1={0x01,0x82,0x03,0x04,0x85,0x06,0x07,0x88};
st=dc_high_disp(icdev,8,numbuff1); //show the result as“12.345.678.”

7) int dc_setbright(int icdev,unsigned char bright);

Description:set the display brightness of the nixietub

Parameter:

icdev: the communication identifier of the device

bright: the brightness value, 0~15 valid,

0:the darkest,

15:the brightest

Return: =0 correct

Example: st=dc_setbright(icdev,10);

8) int dc_settime(int icdev,unsigned char *time);

Description:: set the Reader's time

Parameter:

icdev: the communication identifier of the device

time: the length is 7 bytes, the format is “year、week、month、day、hour、minute、second”

Return: =0 correct

Example: int st;
unsigned char datetime[7]={0x04,0x01,0x04,0x19,0x16,0x35,0x10};
st=dc_settime(icdev,datetime);

9) int dc_srd_eeprom(HANDLE icdev,int offset,int lenh,unsigned char* rec_buffer)

Description: read EEPROM

Parameters:

icdev: the communication identifier of the device
offset: offset address (0-383)
length: the length of data(1-384)
rec_buffer: stores data that read from the EEPROM

Return: = 0: correct
<>0: error

Example: unsigned char Send_buffer[384];
st=dc_srd_eeprom(icdev,0,384,send_buffer);

10) int dc_swr_eeprom(HANDLE icdev,int offset,int lenth,unsigned char *send_buffer)

Description: write EPROM

Parameters:

icdev: the communication identifier of the device
offset: offset address (0-383)
length: the length of data (1-384)
send_buffer: stores data that to write into the EEPROM, the length must less than 384

Return: =0: correct
<>0: error

Example: unsigned char str[6]="Hello";
st=dc_swr_eeprom(icdev,0,5,str);

11) __int16 a_hex(unsigned char *a,unsigned char *hex,__int16 len)

Description:string transform function, transform hex to normal character (long to short).

Parameters:

a : the character that to transformed
hex: the character that has transformed
len: the length of a

Return: =0 correct

12) void hex_a(unsigned char *hex,unsigned char *a,__int16 len)

Description: string transform function,transform normal character to hex (short to long)。

Parameters:

hex: the character that to transformed
a : the character that has transformed
len: the length of hex

Return: =0 correct

13) __int16 __stdcall dc_reset(Handle icdev,unsigned __int16 _Msec)

Description: radio frequency reset function of the device

Parameters:

Icdev: the communication identifier of the device
-Msec: time of reset (ms)

0:close the radio frequency

1,2,...: reset 1,2,...ms

Return: =0 correct

Example: int st;

St=dc_reset(icdev,2)

SHC1102 Card Special Function:

1) int dc_request_shc1102(int icdev,unsigned char _Mode,unsigned int *TagType);

Description: request card

Parameters:

icdev: the communication identifier of the device

_Mode: =0

Tagtype: card type, 0x3300 is shc1102 card

Return: =0 correct

=1 no card

2) int dc_auth_shc1102(int icdev,unsigned char *password)

Description: authentication card

Parameters:

icdev: the communication identifier that the dc_init function returns

password: key, length is 4 bytes

Return: =0 correct

<0 error

3) int dc_read_shc1102(int icdev,unsigned char _Adr,unsigned char *_Data);

Description: read data from the SHC1102 card

Parameters:

icdev: the communication identifier of the device

_Adr: Block Address (0~15);

_Data: stores the data that read from the card, length is 4 bytes

Return: =0 correct

<0 error

Example: int st;

unsigned char data[5];

st=dc_read_shc1102(icdev,4,data); //read data of Block4 from the SHC1102card

4) int dc_write_shc1102(int icdev,unsingned char _Adr,unsigned char *_Data);

Description: write data into the SHC1102 card

Parameters: icdev: the communication identifier of the device

_Adr: Block Address (2~15)

_Data: stores the data that written into the card,4 bytes

Return: =0 correct

<0 error

Mifare Ultralight Special Function:

1) **int dc_anticoll2(int icdev,unsigned char _Bcnt,unsigned long *_Snr);**

Description: anti-collision, returns the last 4 bytes serial number of the card

Parameters:

icdev: the communication identifier of the device

_Bcnt: set 0

_Snr: databuff that stores the serial number of the card

Return: =0 correct

<0 error

Example: int st;

unsigned long snr;

st=dc_anticoll2(icdev,0,&snr);

2) **int dc_select2(int icdev,unsigned long _Snr,unsigned char *_Size);**

Description: selects the card with the specified serial number(the last 4 bytes) from multiple cards.

Parameters: icdev: the communication identifier of the device

_Snr: the serial number of the card(dc_anticoll2 returns)

_Size: the memory size of the card

Return: =0 correct

<0 error

Example: int st,type;

unsigned char size;

unsigned long snr;

dc_anticoll2(icdev,0,&snr);

st=dc_select2(icdev,snr,&size);

ISO14443-4 Card Special Function:

1) **__int16 __stdcall dc_config_card(HANDLE icdev,unsigned char cardtype);**

Description: Set the card type that the Reader is about to operate, when the Reader is powered on ,the default status is to operate the type A card.

Parameters:

Icdev: the communication identifier that the dc_init returns

Cardtype: when the cardtype is 'A', that's set the Reader, when 'B', that's operate the type B card'

Return: =0 correct

<0 error

Example:

int st;

st= dc_config_card (icdev,'B'); //operate the Type B card//

2) **__int16 dc_pro_command(HANDLE ICDev, unsigned char slen,unsigned char * sbuff,unsigned char *rlen,unsigned char * rbuff,unsigned char tt)**

Description: the information exchange function of the APDU, this function has sealed the T=CL operation

Parameters:

icdev: the communication identifier that dc_init returns

slen: length of the sending information

sbuff: databuffer that stores the sending information

rlen: length of the information that returns

rbuff: databuffer that stores the returning information

tt: delay time, the unit is 10ms

Return: <0 error ,its absolute value is the error error

=0 correct

Example: int st;

```
unsigned char slen,rlen,snddata[100], recdata[100];
```

```
slen=5;
```

```
senddata[0]=0x00;senddata[1]=0x84;senddata[2]=0x00;senddata[3]=0x00;
```

```
senddata[4]=0x04;
```

```
st= dc_pro_command ( icdev,slen,senddata,&rlen,recdata,7)
```

Related Function:

```
__int16 __stdcall dc_pro_command_hex(HANDLE idComDev,unsigned char slen, char *  
sendbuffer,unsigned char *rlen, char * databuffer,unsigned char timeout)
```

3) __int16 dc_pro_commandsource(HANDLE ICDev,unsigned char slen,unsigned char * sbuff,unsigned char *rlen,unsigned char * rbuff,unsigned char timeout)

Description: information exchange function of APDU. This function doesn't seal. The user needs to organized the data transmission voluntarily

Parameters:

icdev: the communication identifier that dc_init returns

slen: length of the sending information

sbuff: databuffer that stores the sending information

rlen: length of the information that returns

rbuff: databuffer that stores the returning information

tt: delay time, the unit is 10ms

Return: <0 error ,its absolute value is the error error

=0 correct

Example: int st;

```
unsigned char slen,rlen,snddata[100], recdata[100]
```

```
slen=5;senddata[0]=nad;senddata[1]=pcb;senddata[2]=5,
```

```
senddata[3]=0x00;senddata[4]=0x84;senddata[5]=0x00
```

```
senddata[6]=0x00;senddata[7]=0x08;
```

```
for(st=0;st<8;st++)
```

```
senddata[8]^=senddata[st];
```

```
st=dc_pro_commandsource( icdev,slen,senddata,&rlen,recdata,7);
```

Related Function:

```
__int16 __stdcall dc_pro_commandsource_hex(HANDLE ICDev,unsigned char  
slen,unsigned char * sbuff,unsigned char *rlen,unsigned char * rbuff,unsigned char  
timeout)
```

4) __int16 dc_pro_commandlink(HANDLE ICDev,unsigned char slen,unsigned char * sbuff,unsigned char *rlen,unsigned char * rbuff,unsigned char tt,unsigned char FG)

Description: the information exchange function of the APDU, this function has sealed the T=CL operation

Parameters:

ICdev: the communication identifier that dc_init returns

slen: length of the sending information

sbuff: databuffer that stores the sending information

rlen:length of the information that returns

rbuff: databuffer that stores the returning information

tt: delay time, the unit is 10ms

FG: division length. Suggested this value is less than 64

Return: <0 error ,its absolute value is the error error
=0 correct

Example: int st;

```
unsigned char slen,rlen,snddata[100], recdata[100];
slen=5;
senddata[0]=0x00;senddata[1]=0x84;senddata[2]=0x00;
senddata[3]=0x00;senddata[4]=0x04;
st= dc_pro_commandlink ( icdev,slen,senddata,&rlen,recdata,7, 56)
```

Related Function:

```
__int16 __stdcall dc_pro_commandlink_hex(HANDLE ICDev, unsigned char
slen,unsigned char *buff, unsigned char *rlen,unsigned char * rbuff,unsigned char
tt,unsigned char FG)
```

5) __int16 dc_pro_reset(HANDLE ICDev,unsigned char *rlen, unsigned char *rbuff)

Description: card's electrify and reset function. only limits to Type A card

Parameters

iCDev: the communication identifier that the dc_init function returns

rlen: length of the reset information returns

rbuff: stores reset information that returns

Return:<0 error ,its absolute value is the error number
=0 correct

Example:st=dc_pro_reset(ICDev,rlen,DataBuffer)

Related Function:

```
__int16 __stdcall dc_pro_reset_hex(HANDLE icdev,unsigned char *rlen, char *rec
eive_data)
```

6) int dc_pro_halt(int icdev)

Description: halt card, only limits to Type A card

Parameters:

icdev: the communication identifier of the device

Return: <0 error

=0 correct

Example: st=dc_pro_halt(icdev);

Attention: when you use the function of dc_card(), there is a parameter Mode, if Mode=0, then after operating on the card, execute the function of dc_pro_halt(), then the card turns into the HALT Mode, here you must move the card out of the operation area and then remove it into this area again, then you can find the card.

7) __int16 __stdcall dc_request_b(HANDLE icdev, unsigned char _Mode, unsigned char AFI, unsigned char N, unsigned char *ATQB);

Description: find Type B card

Parameters:

icdev: the communication identifier that the dc_init function returns

Mode: reservation parameter, set 0

N: the main parameter of the anti_collision, used to calculate the time_idle number

AFI: Application Family Identifier

ATQB: stores the information that returns

Return: <0 error

=0 correct

8) __int16 __stdcall dc_attrb(HANDLE icdev, unsigned char *PUPI, unsigned char CID);

Description: attribute the Type B card

Parameters:

icdev: the communication identifier that the dc_init function returns

PUPI: the main parameter of attrb, go to the reference 1443-3

CID: value from 0 to 14, if the card supports CID, it can be used to specify and partition the card

Return: <0 error

=0 correct

9) __int16 __stdcall dc_card_b(HANDLE icdev, unsigned char *rbuf);

Description: request Type B card

Parameter:

icdev-- the communication identifier that the dc_init returns

*rbuf—the reset information when request Type B card OUT

Return:

0--success

1---No card

Others--error

Related function:

__int16 __stdcall dc_card_b_hex(HANDLE icdev, char *rbuf);

Notice: Can use the dc_pro_commandlink directly after using this function.

10) __int16 __stdcall dc_cardAB(HANDLE icdev,unsigned char *rlen,unsigned char *rbuf,unsigned char *type);

Description: request TypeA card or TypeB card

Parameter:

icdev-- the communication identifier that the dc_init returns

*rlen—the length of the reset information OUT

*rbuf—reset information OUT

*type—card type, returns A means TypeA card; returns B means TypeB card OUT

Return:

0--success

1—No card

others--error

Mifare Des Card Special Function:

1) int dc_antcoll2(int icdev,unsigned char _Bcnt,unsigned long *_Snr);

Description: anti-collision, returns the last 4 bytes serial number of the card

Parameters:

icdev: the communication identifier of the device

_Bcnt: set 0

_Snr: databuff that stores the serial number of the card

Return: =0 correct

<0 error

Example: int st;

unsigned long snr;

st=dc_antcoll2(icdev,0,&snr);

2) int dc_select2(int icdev,unsigned long _Snr,unsigned char *_Size);

Description: selects the card with the specified serial number(the last 4 bytes) from multiple cards.

Parameters: icdev: the communication identifier of the device

_Snr: the serial number of the card(dc_antcoll2 returns)

_Size: the memory size of the card

Return: =0 correct

<0 error

Example: int st,type;

unsigned char size;

unsigned long snr;

dc_antcoll2(icdev,0,&snr);

st=dc_select2(icdev,snr,&size);

3) int dc_pro_reset (HANDLE ICDev,unsigned char *rlen, unsigned char *rbuff)

Description: electrify and reset the card

Parameters:

icdev: the communication identifier that dc_init returns

rlen: length of the reset information

rbuff: stores the reset information that returns

Return: <0 error ,its absolute value is the error code
=0 correct

Example: st=dc_pro_reset(ICDev,rlen,DataBuffer)

Related Function:

__int16 __stdcall dc_pro_resethex(HANDLE icdev,unsigned char *rlen, char *receive_data)

4) int dc_pro_halt(HANDLE icdev);

Description: stop operating on the card

Parameters

icdev: the communication identifier of the device

Return: =0: correct
<>0: error

Example: st=dc_pro_halt(icdev);

Attention: when you use the function of dc_card(), there is a parameter Mode, if Mode=0, then after operating on the card, execute the function of dc_pro_halt(), then the card turns into the HALT Mode, here you must move the card out of the operation area and then remove it into this area again, then you can find the card.

dc_pro_command, dc_pro_commandsource, dc_pro_commandlink ,please see the introduction in ISO14443-4

CPU(SAM) Card Special Function:

1) __int16 dc_cpureset(HANDLE ICDev,unsigned char *rlen, unsigned char *rbuff)

Description: electrify and reset function of the CPU card, it can judge the protocol of the card automatically after reset.

Parameters:

icdev: the communication identifier that dc_init returns

rlen: length of the information that returns

rbuff: databuffer that stores the returning information

Return: <0 error, its absolute value is the error code
=0 correct

Example:

st=dc_cpureset(ICDev,rlen,DataBuffer)

Related function:

__int16 __stdcall dc_cpureset_hex(HANDLE icdev,unsigned char *rlen, char *databuffer)

2) __int16 dc_setcpu(HANDLE ICDev,unsigned char SAMID)

Description: set the SAM card seat that to operate

Parameter:

icdev: the communication identifier that dc_init returns

SAMID:set the card seat number that to operate,0x0c is the attached card seat,
 0x0d ,0x0e ,0x0f is the SAM1,SAM2,SAM3 respectively
 Return: <0 error ,its absolute value is the error code
 =0 correct

3) **__int16 dc_cpuapdu(HANDLE ICDev,unsigned char slen,unsigned char * sbuff,unsigned char *rlen,unsigned char * rbuff)**

Description:the information exchange function of APDU of the CPU card,this function has
 sealed the T=0 and the T=1 operation.

Parameters:

Icdev: the communication identifier that dc_init returns

slen: length of the sending information

sbuff: databuffer that stores the sending information

rlen:length of the information that returns

rbuff: databuffer that stores the returning information

Return: <0 error, its absolute value is the error code
 =0 correct

Example:int st;

```
unsigned char slen,rlen,snddata[100], recdata[100];
slen=5;
senddata[0]=0x00;senddata[1]=0x84;senddata[2]=0x00;
senddata[3]=0x00;senddata[4]=0x04;
st= dc_cpuapdu ( icdev,slen,senddata,&rlen,recdata)
```

Related Function:

**__int16 __stdcall dc_cpuapdu_hex(HANDLE icdev,unsigned char slen, char *
 sendbuffer,unsigned char *rlen,unsigned char * rbuff)**

4) **__int16 dc_cpuapdusource(HANDLE ICDev,unsigned char slen,unsigned char * sbuff,unsigned char *rlen,unsigned char * rbuff)**

Description: information exchange function of APDU. This function doesn't seal. The user
 needs to organized the data transmission voluntarily

Parameters:

Icdev: the communication identifier that dc_init returns

slen: length of the sending information

sbuff: databuffer that stores the sending information

rlen:length of the information that returns

rbuff: databuffer that stores the returning information

Return: <0 error ,its absolute value is the error error
 =0 correct

Example: T=1 protocol card function

```
int st;
unsigned char slen,rlen,snddata[100], recdata[100];
slen=5;senddata[0]=nad;senddata[1]=pcb;senddata[2]=5,
senddata[3]=0x00;senddata[4]=0x84;senddata[5]=0x00;
```

```

senddata[6]=0x00;senddata[7]=0x08;
for(st=0;st<8;st++)
senddata[8]^=senddata[st];
st=dc_cpuapdusource( icdev,slen,senddata,&rlen,recdata);

```

Related Function:

```

__int16 __stdcall dc_cpuapdusource_hex(HANDLE icdev,unsigned char slen, char *
sendbuffer,unsigned char *rlen,unsigned char * rbuff)

```

5) **__int16 dc_setcpupara(HANDLE ICDev,unsigned char cputype,unsigned char cpupro,unsigned char cputu)**

Description: set CPU card's parameter, the default parameter is cpupro=0(T=0 protocol) cputu=92(baudrate 9600) after electrifying.

Parameter:

ICDev :the communication identifier that dc_init returns

ccputype: the type of the card seat

12=main card seat

13=SAM1 card seat

14=SAM2 card seat

15=SAM3 card seat (decimalist)

cpupro: the protocol type of the card

=0:T=0 protocol

=1:T=1 protocol

cputu : the delay data (decimalist) of the card when operating.With different cards,this parameter's value is different ,For the card whose baudrate is 9600, set cputu as 92 . For the card whose baudrate is 38400,set cputu as20.

Return: <0 error, its absolute number is the error code

=0 correct

FM11RF005 Card Special Function:

1) **int dc_read_fm11rf005(int icdev,unsigned char _Adr,unsigned char *_Data);**

Description: read data of the FM11RF005 card

Parameters:

icdev: the communication identifier of the device

_Adr: Block Address (0~15) ;

_Data: read data, the length is 4 bytes

Return: = 0: correct

<>0: error

Example:

```

int st;
unsigned char data[5];
st=dc_read_fm11rf005(icdev,4,data);

```

Related Functions:

```

int dc_read_fm11rf005_hex(int icdev,unsigned char _Adr,char *_Data);

```

2) int dc_write_fm11rf005(int icdev,unsigned char _Adr,unsigned char *_Data);

Description: write data into the FM11RF005 card

Parameters:

icdev: the communication identifier of the device

_Adr: Block Address (2~15) ;

_Data: write data, the length is 4 bytes

Return: = 0: correct

<>0: error

Example:

```
int st;
```

```
unsigned char data[5];
```

```
st=dc_write_fm11rf005(icdev,4,data);
```

Related Functions:

```
int dc_write_fm11rf005_hex(int icdev,unsigned char _Adr,char *_Data);
```

3)int dc_getsnr_fm11rf005(int icdev, unsigned long *_snr);

Description: read the fm11rf005 card's serial number

Parameters:

icdev: the communication identifier of the device

buff: returns card's serial number

Return: = 0: correct

<>0: error

Example: int st;

```
unsigned char buff[20];
```

```
st=dc_getsnr_fm11rf005(icdev,buff);
```

Related Function:

```
int dc_getsnr_fm11rf005_hex(int icdev, unsigned char *snrstr);
```

T8 LCD Display Function:

1.dc_dispinfo_T8(HANDLE idComDev,unsigned char line,unsigned char offset,char *data)

Description:T8 LCD display function, line No. included

Parameters:

HANDLE idComDev-- the communication identifier of the device

unsigned char line----line No(0--3) IN

unsigned char offset----offset(0--7) IN

char *data---the data to display IN

Return:

0--success

Notice: (offset+Length of data)<=8

Example:

```
int st;
```

```
char data[5]={0x31,0x32,0x33,0x34,0};
```

```
st=dc_dispinfo_T8(icdev,0,4,data);
```

```
st=dc_dispinfo_T8(icdev,1,1,"中华人民共和国");
```

2.dc_dispinfo_pro_T8(HANDLE idComDev,unsigned char offset,char *data)

Description: T8 LCD display function,line No. not included.

Parameters:

HANDLE idComDev-- the communication identifier of the device

unsigned char offset---offset(0--31) IN

const char *data---data to display IN

Return:

0--success

Example:

```
int st;
```

```
st=dc_dispinfo_pro_T8(icdev,25,"0123456");
```

3.__int16 __stdcall dc_clearlcd_T8(HANDLE icdev,unsigned char line);

Description:clear LCD

Parameters:

icdev-- the communication identifier of the device

line--

0: clear line 0

1: clear line 1

2: clear line 2

3: clear line 3

4: clear all lines

return:0--success

Function Error Code
return instruction

Win32 Function Error Code:

Return value (negative)	Error type
0x10	Communication error
0x11	Overtime error
0x20	Open port error
0x21	Get port parameter error
0x22	Set port parameter error
0x23	Close port error
0x24	Port impropriated
0x30	Format error
0x31	Data format error
0x32	Data length error
0x40	Read error
0x41	Write error
0x42	Non_receive error
0x50	No reduce error
0x51	CPU data verify error
0x52	Number of address error when RS485 communicating
0x73	Get the version number error
0xc2	CPU card response error
0xd3	CPU card response overtime
0xd6	CPU card verify error
0xd7	CPU card command character error

Return value (positive number)	Error type
1	No card or verify error
2	Data verify error
3	Value vacancy error
4	Authentication error
5	Parity verify error
6	Reader and card communicating error
8	Read Card serial number error
9	Key type error
10	Card hasn't verified
11	To operate card bits error
12	To operate card bytes error
15	Write card error
16	Increment operate fail
17	Decrement operate fail
18	Read Card operate fail
19	Transmission buffer overflow
21	Transmission frame error
23	Unknown transmission requirement
24	Anticollision error
25	RF module reset error
26	Non-verify interface
27	RF module communication overtime
60	Non-normal operation
100	Wrong data
124	Wrong parameter value